

051-文件系统

{% hint style="warning" %} 本章 ppt 过于混乱，尽量按照人能理解的顺序重排了..... {% endhint %}

文件 & 文件系统

- 文件：具有符号名的，在逻辑上具有完整意义的一组相关信息项的序列
- 文件名 + 拓展名
- 优点：对数据本身的封装，使用方便、安全可靠、可备份、可共享
- 文件系统：负责存取、管理信息的模块
 - 文件系统文件：逻辑结构 + 物理结构
 - 功能
 - 文件的按名存取
 - 文件的共享和保护
 - 文件的操作和使用
 - 组成
 - 文件组织
 - 文件存取：顺序、直接、索引
 - 文件控制：逻辑 & 物理
 - 文件使用：打开、关闭、读、写、控制

文件组织

文件存储

- 卷：存储介质的物理单位，一盘磁带、一个磁盘分区
- 块：介质上连续信息组成的区域，也叫做物理记录
 - 主存和外存**以块为单位**交换信息
 - 相邻块之间存在间隙
- 顺序存取：严格按照信息的物理位置进行定位、存取，磁带，光盘
- 直接/随机存取：定位信息的时间不依赖于物理位置，磁盘

GPT 说：现代磁盘没有物理间隙，磁带等顺序介质上相邻块之间存在记录间隙；磁盘上块是逻辑连续编号。

文件的逻辑结构

- 流式文件：无结构，无固定分界，通过长度/分界符读取信息
 - 日志文件、文本文件
- 记录文件：有结构，以记录为基本单位
 - 某些二进制文件
 - 数据库系统：更复杂，存在冗余

记录的成组 & 分解

- 逻辑记录：记录文件内的一个独立数据单元（一条信息）
- 物理记录：块

- 记录的成组：当多个逻辑记录存储在同一个物理记录中时，以块为单位进入 Buffer
- 记录的分解：在 Buffer 中将物理记录分解为逻辑记录
- 记录方式
 - 定长记录：在块末存在未使用的内部碎片
 - 变长记录：大多数块都有未使用空间
 - 跨块记录：跨块记录间使用索引/链表结构
- 优点：减少 I/O 操作，提高存取效率
 - 提前读：读一个块时，读入了多个逻辑记录
 - 推迟写：等到块中的所有逻辑记录都被修改后，再写入磁盘

文件的物理结构

结构种类	定义	优点	缺点
顺序文件	物理记录按顺序存储	存取速度快	修改、插入、增加困难
连接文件	物理记录不连续，块末有连接字指向下一块	插入、删除、修改方便	存取速度慢，连接字开销
直接文件	根据关键字 Hash 计算物理地址	存取速度快	冲突处理复杂

索引文件

- 在文件的开头建立索引表，索引表中存储了每个逻辑记录的**键 + 物理地址**
- 存储器分区：索引区 + 数据区
- 索引表地址由**目录**给出
- 读文件步骤：读索引表 -> 查表获取物理地址 -> 读数据区
- 将索引表调入内存减少 I/O 操作
- 优点：连接结构的拓展，便于增删改
- 缺点：索引表的时空开销
- 多级索引表
 - 直接块：直接指向数据块
 - 间接块：指向的索引表内的块编号范围为 [该块编号, 下一块编号 - 1]

在UNIX系统中，每个i节点中分别含有10个直接地址的索引和一、二、三级间接索引。若每个盘块存放128个盘块地址，则一个1MB的文件分别占用多少各级索引所使用的数据物理块？20MB的文件呢？设每个盘块有512B。

- 直接块容量： $10 \cdot 512B = 5KB$
- 一级间接块容量： $128 \cdot 512B = 64KB$
- 二级间接块容量： $128 \cdot 128 \cdot 512B = 8MB$
- 三级间接块容量： $128 \cdot 128 \cdot 128 \cdot 512B = 1GB$

1MB 文件 占用：

- 直接块：10
- 一级间接块：128

- 二级间接块: $\frac{1024 - 69}{512} = 1910$
- 三级间接块: 0

20MB 文件 占用:

- 直接块: 10
- 一级间接块: 128
- 二级间接块: 16384
- 三级间接块: $\frac{20480 - 69 - 8192}{512} = 24438$

文件目录

- 实现“按名存取”的关键数据结构
- 目录本身也是文件
- 一级目录结构: 线性表、容易重名
- 二级目录结构: 主文件目录 + 用户文件目录
 - 用户仅能访问自己的文件目录
 - naive 的权限管理
 - 用户目录下混乱
- 树形目录结构
 - Windows 将每棵子树对应一个磁盘分区
 - Linux 将每棵子树挂载到一个目录下

文件目录管理

- `cd`: 改变当前目录
 - `.`: 当前目录
 - `..`: 上级目录
- 查找目录项: 顺序/二分/Hash
- 文件目录处理
 - 沿路径查找: 速度慢
 - 将所有目录项读入内存: 主存开销大
 - Tradeoff: 把常用、使用中的目录项读入内存

文件保护

- 共享时的并发控制
- 保密
- 文件保护
 - 动态多副本: 在多个介质上同时存储同一文件, 故障时切换 (小型重要文件)
 - 文件转储: 定期备份
 - 文件存取控制矩阵
 - 行: 用户
 - 列: 文件
 - 权限: 读、写、执行
 - 存取控制表: 用户|文件|权限
 - 消除矩阵冗余, 仅登记对文件拥有存取属性的部分
- 文件属性

- 用户分类：所有者、组用户、其他用户
- 权限分类：读、写、执行
- `chmod`：改变文件权限
- `chown`：改变文件拥有者
- `chgrp`：改变文件组

文件存取

- 与文件物理结构匹配
- 顺序存取：根据读/写指针读写，完成后推进指针，可前进/后退
- 直接存取：直接根据 Hash 函数计算出存储位置并读写
- 索引存取：根据 Key 计算出索引位置

文件使用

- 使用命令 / 系统调用
- 文件句柄：打开文件后返回的标识符，基于该标识符对文件进行操作

操作	参数	流程	备注
建立文件	文件名、设备类/号、文件属性、存取控制信息	为新文件分配索引节点和活动索引节点，并把索引节点编号与文件分量名组成新目录项，记到目录中。 在新文件所对应的活动索引节点中置初值，如置存取权限<i>i_mode</i>，连接计数<i>i_nlink</i>等。 分配用户打开文件表项和系统打开文件表项，置表项初值，读写位移<i>f_offset</i>清零。 把各表项及文件对应的活动索引节点用指针连接起来，把文件描述字返回给调用者。	<pre>int create(char *filename, int mode)</pre>
撤销文件 (删除)	文件名，设备类/号	在目录文件中删去相应目录项； 将引用计数减1，若减为0则释放文件占用的文件存储空间	<pre>int delete(char *filename)</pre>
打开文件	文件名，设备类/号，打开方式	检查目录，把外存索引节点复制到 活动inode表 检查权限 分配并初始化用户打开文件表项、系统打开文件表项 返回文件描述符	<pre>int open(char *filename, int mode)</pre>

操作	参数	流程	备注
关闭文件	文件句柄	根据fd找到用户打开文件表项，再找到系统打开文件表项，释放用户打开文件表项 在活动inode表中将对应的i_count减1，若减为0则写回并释放inode； 在系统打开文件表将对应的f_count减1，若减为0则释放表项，完成“延迟写”；若目录项修改则写入目录文件	<pre>void close(int fd)</pre>
读取文件	文件句柄，用户数据区地址，长度	检查权限； 将逻辑地址转换为物理地址，根据f_offset和len找到读入位置 从磁盘读数据到内核缓冲区； 将数据从内核缓冲区复制到用户数据区并返回结果	<pre>int read(int fd, char *buf, int len)</pre>
写入文件	文件句柄，用户数据区地址，长度	检查权限； 将逻辑地址转换为物理地址，根据f_offset和len找到写入位置 将数据从用户数据区复制到内核缓冲区； 将数据从内核缓冲区写入磁盘	<pre>int write(int fd, char *buf, int len)</pre>
定位文件 (调整读写指针位置)	文件句柄，偏移量， whence: 0-从文件头开始，1-从当前位置开始	根据偏移量调整读写指针位置	<pre>int lseek(int fd, int offset, int whence)</pre>

辅存空间管理

- 碎片：文件不断建立撤销后，磁盘上产生大量空闲空间
- 碎片整理：重新组织文件，使其连续
- 辅存空间分配方式
 - 连续分配：顺序访问快、管理简单、需碎片整理
 - 非连续分配：便于文件增长收缩
- 管理空闲块
 - Bitmap
 - 空闲块成组连接法：使用链表，分组管理空闲块，每个链表的第一块指向下一链表
- 格式化磁盘后系统可用空间变小：bitmap和inode表项都需要空间

文件系统实现层次

- 用户接口：接受系统调用
- 逻辑文件控制子系统：维护活动文件表，将逻辑记录转化为物理块号

- 文件保护子系统：识别身份，验证权限
- 物理文件控制子系统：管理 buffer、分配存储空间
- I/O 控制子系统：具体实现 I/O 操作